



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Minimizing Cycle Time in a Blocking Flowshop

I.N. Kamal Abadi, Nicholas G. Hall, Chelliah Sriskandarajah,

To cite this article:

I.N. Kamal Abadi, Nicholas G. Hall, Chelliah Sriskandarajah, (2000) Minimizing Cycle Time in a Blocking Flowshop. Operations Research 48(1):177-180. <https://doi.org/10.1287/opre.48.1.177.12451>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 2000 INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

MINIMIZING CYCLE TIME IN A BLOCKING FLOWSHOP

I.N. KAMAL ABADI

Faculty of Engineering, University of Kordestan, Sanandaj, Islamic Republic of Iran

NICHOLAS G. HALL

Fisher College of Business, The Ohio State University, 2100 Neil Avenue, Columbus, Ohio 43210, hall.33@osu.edu

CHELLIAH SRISKANDARAJAH

School of Management, The University of Texas at Dallas, 2601 N. Floyd Road, Richardson, Texas 75083-0688, chelliah@utdallas.edu

(Received September 1995; revision received December 1997; accepted July 1998)

We consider a blocking (i.e., bufferless) flowshop that repetitively processes a minimal part set to minimize its cycle time, or equivalently to maximize its throughput rate. The best previous heuristic procedure solves instances with 9 machines and 25 jobs, with relative errors averaging about 3% but sometimes as much as 10%. The idea of deliberately slowing down the processing of operations (i.e., increasing their processing times) establishes a precise mathematical connection between this problem and a no-wait flowshop. This enables a very effective heuristic for the no-wait flowshop to be adapted as a heuristic for the blocking flowshop. Our computational results show relative errors that average less than 2% for instances with 20 machines and 250 jobs.

We consider a *blocking* flowshop manufacturing environment with m machines arranged in series. There are k job types. A job consists of m operations, to be performed on the machines in the sequence $M_1, \dots, M_m$. The processing time of an operation of job type j on machine M_i is denoted $p_{i,j}$. The flowshop has no intermediate buffers; therefore a job cannot leave a machine until the next machine downstream is free. The machines are required to produce repetitively a minimal part set, or MPS, which consists of r_j units of job type j , for $j = 1, \dots, k$, where $r_1 + \dots + r_k = n$. The objective is the minimization of the steady state cycle time to produce an MPS, or equivalently the maximization of throughput rate. A variety of scheduling environments with blocking are discussed by Hall and Sriskandarajah (1996).

Scheduling models with blocking gain considerable attention from the study of McCormick et al. (1989), who consider an m machine flowshop with fixed capacity buffers between machines, which is equivalent to our problem. Eight instances with 10 machines and 80 jobs are solved heuristically, but comparisons with optimal cycle times or lower bounds are not provided. Karabati and Kouvelis (1996) develop an integer programming model for the blocking flowshop problem. A branch-and-bound algorithm solves problem instances with 10 machines and 12 jobs to optimality within a few minutes of IBM 3081 CPU time. For problems with 9 machines and 25 jobs, heuristics deliver solutions with relative errors that average less than 3% but can exceed 10%.

1. PROBLEM DEFINITION

Because of the absence of buffers in a blocking flowshop, no job can overtake another. We therefore seek a *permutation* schedule, where the job sequence is the same on all machines. Let

$$\delta_{ij} = \begin{cases} 1 & \text{if job } j \text{ is in position } i \text{ in the sequence,} \\ 0 & \text{otherwise,} \end{cases}$$

s_i^k = the starting time of the operation on machine M_k of the job in position i ,

$C_B(\delta)$ = the cycle time for the flowshop with the jobs in sequence δ .

We can now formulate the blocking flowshop problem (problem B) as follows:

$$C_B^* = \min_{\delta \in \Delta} C_B(\delta) \quad (1)$$

$$\text{s.t. } C_B(\delta) \geq s_{n+1}^k - s_1^k, \quad k = 1, \dots, m, \quad (2)$$

$$s_i^{k+1} \geq s_i^k + \sum_{j=1}^n p_{k,j} \delta_{ij}, \quad i = 1, \dots, n, \quad k = 1, \dots, m-1, \quad (3)$$

$$s_{i+1}^k \geq s_i^k + \sum_{j=1}^n p_{k,j} \delta_{ij}, \quad i = 1, \dots, n, \quad k = 1, \dots, m, \quad (4)$$

Subject classifications: Production/scheduling: sequencing, blocking flowshop. Manufacturing: Traveling salesman problem.

Area of review: LOGISTICS AND SUPPLY CHAIN OPERATIONS.

$$s_{i+1}^k \geq s_i^{k+1}, \quad i = 1, \dots, n, \quad k = 1, \dots, m-1, \quad (5)$$

$$\sum_{i=1}^n \delta_{ij} = 1, \quad j = 1, \dots, n, \quad (6)$$

$$\sum_{j=1}^n \delta_{ij} = 1, \quad i = 1, \dots, n, \quad (7)$$

$$\delta_{ij} \in \{0, 1\}, \quad i, j = 1, \dots, n, \quad (8)$$

$$s_i^k \geq 0, \quad i = 1, \dots, n, \quad k = 1, \dots, m, \quad (9)$$

where $n+1=1$ and $\delta = \{\delta_{ij}, i, j = 1, \dots, n\}$. Let Δ denote the set of all possible sequences δ and associated starting times of operations that satisfy constraints (2)–(9).

In this formulation, the objective (1) and constraints (2) ensure that the cycle time of the flowshop is equal to the cycle time of the bottleneck machine. Constraints (3) ensure that a job cannot start processing on a machine until it has finished processing on the previous machine. Constraints (4) require a machine to have finished processing a job before it starts processing the next job. Constraints (5) require a job to be started on a machine no sooner than the previous job is started on the next machine downstream, thereby enforcing the blocking condition. Finally, constraints (6) and (7) require each job to occupy exactly one position in the sequence, and each position to accept exactly one job.

2. EQUIVALENCE OF BLOCKING AND NO-WAIT FLOWSHOPS

Consider a relaxation of problem B, which we call problem BS, where we allow *slowing down* (i.e., increasing the processing times of) operations. Also closely related to problem B is the no-wait flowshop problem, N. Problem N can be described using formulation B, with the only difference that (3) is replaced by

$$s_i^{k+1} = s_i^k + \sum_{j=1}^n p_{k,j} \delta_{ij}, \quad i = 1, \dots, n, \quad k = 1, \dots, m-1. \quad (10)$$

In the no-wait flowshop N, the slowing down of some operations may permit a reduction in cycle time. Finally, we define problem NS, which is similar to problem BS but has (3) replaced by (10). We denote the optimal cycle time for a given instance of problem BS (respectively, N, NS) by C_{BS}^* (resp., C_N^* , C_{NS}^*).

THEOREM 1. *Given any sequence δ , $C_B(\delta) = C_{NS}(\delta)$.*

PROOF. Consider any instance of problem B and any given job sequence $\delta \in \Delta$. In the formulation of problem B, the processing times appear only in constraints (3) and (4). From the direction of those inequalities, we have $C_{BS}(\delta) = C_B(\delta)$. Also, problem NS is a restriction of problem BS in which the inequality in constraints (3) is changed

into an equality, thus $C_{BS}(\delta) \leq C_{NS}(\delta)$. Combining this with the previous result gives $C_B(\delta) \leq C_{NS}(\delta)$.

We now show how, given a sequence δ with cycle time $C_B(\delta)$ for problem B, we can construct a schedule for problem NS with cycle time $C_{NS}(\delta) = C_B(\delta)$. In the schedule for problem B, all the constraints of problem NS are satisfied, except that some constraints of (3) may be strict inequalities. That is, there may exist some i , $1 \leq i \leq n$, and k , $1 \leq k \leq m-1$, such that $s_i^{k+1} > s_i^k + \sum_{j=1}^n p_{k,j} \delta_{ij}$. If doing so is feasible, any such inequality can be converted into an equality by setting $p_{k,j} = s_i^{k+1} - s_i^k$ for all jobs $\{j | \delta_{ij} = 1\}$. This change increases the processing time and completion time on machine M_k of the job in position i , but none of the start times of operations change. Now, the feasibility of inserting the additional processing time requires that machine M_k is idle throughout the time interval $[s_i^k + \sum_{j=1}^n p_{k,j} \delta_{ij}, s_i^{k+1}]$. However, this condition is equivalent to $s_{i+1}^k \geq s_i^{k+1}$, as already enforced by constraints (3) and (5). Thus $C_{NS}(\delta) \leq C_B(\delta)$. $\square$

3. ALGORITHM FOR SLOWING DOWN

We consider an instance of problem B in which the job sequence is given and no slowing down has occurred. In order to take advantage of the link between problems B and NS established by Theorem 1, we need an algorithm that allows the slowing down of operations by taking data for B as input and delivering a no-wait schedule for NS as output. For this reason, the network flow algorithm of McCormick et al. (1989), which delivers a blocking schedule that is not in general no-wait, is not adequate for our purpose. The main result of this section now follows.

THEOREM 2. *Given a job sequence and a corresponding no-wait schedule, the problem of finding the minimum cycle time no-wait schedule by slowing down operations can be formulated as a minimum cost flow problem in a network.*

PROOF. See Abadi et al. (1997), available at the Operations Research Home Page: <http://opim.wharton.upenn.edu/~harker/opsresearch.html>.

4. HEURISTICS

We describe and test two heuristic procedures for problem B. It follows from Theorems 1 and 2 that a possible strategy for solving problem B is to find a good job sequence for problem N, and then slow down the operations. Note that the amount of slowing down of an operation of a job in NS is equivalent to the amount of blocking time in B of that job on the corresponding machine processing the operation. Piehler (1960) shows that problem N is equivalent to a traveling salesman problem (TSP). A tour in the TSP represents a sequence of jobs in the scheduling problem. While the TSP is formally intractable (Garey and Johnson 1979, p. 211), there exist very effective heuristics, such as GENIUS (Gendreau

et al. 1992), for solving it. GENIUS contains procedures GENI and US. To create an initial sequence, GENI starts with a subsequence of three arbitrary jobs. The other jobs are then taken one at a time in arbitrary order and inserted in the best position in the current subsequence. To find an improved sequence, US removes jobs one at a time from the sequence and inserts them in the best position by trying n possible positions. Thus, each US operation creates a new sequence in the neighborhood of the best previous sequence.

We now describe a heuristic that uses these ideas for problem B. In the algorithm described below, $NF(\delta)$ denotes a subroutine that applies a network flow algorithm to the formulation of Theorem 2 for a given instance of problem B and a given job sequence δ , to obtain an optimal solution for problem NS. The algorithm for finding a minimum cost flow in a network which is implemented in SSD is by Orlin (1993). In general, Step 1 of SSD finds a sequence σ' with a cycle time that is a good upper bound. Then, by n repetitions of Step 2, SSD tries to improve this upper bound by searching among the sequences in the neighborhood of σ' .

HEURISTIC SEQUENTIAL SLOWING DOWN (SSD)

Step 0. Input the matrix $P = \{p_{k,j}, k = 1, \dots, m, j = 1, \dots, n\}$ and the matrix $D = \{d_{ij}, i, j = 1, \dots, n\}$, where $d_{ij} = \max_{i \leq k \leq m} \{\sum_{h=1}^k p_{h,i} - \sum_{h=1}^{k-1} p_{h,j}\}$ denotes the cycle time contribution if job j follows job i in problem N.

Step 1. Use GENI to find a sequence σ for problem N. Set $\sigma^* = \sigma$ and $j = 1$. Call $NF(\sigma)$, and set $\sigma^* = \sigma' = \sigma$ and $z^* = C_B(\sigma)$.

Step 2. Starting from sequence σ' , use US and GENI with job j to find a sequence σ'' . Call $NF(\sigma'')$. If $C_B(\sigma'') < z^*$, then update P and D and set $\sigma^* = \sigma' = \sigma''$ and $z^* = C_B(\sigma^*)$.

Step 3. If $j = n$, then output sequence σ^* and the cycle time z^* , and terminate. Otherwise, set $j = j + 1$ and go to **Step 2**.

Unfortunately, the two-phase strategy of heuristic SSD may not perform well on some problem instances. It may happen that the heuristic job sequence delivered by GENIUS provides a no-wait schedule that is capable of little or no cycle time reduction through the slowing down of operations. However, other neighboring sequences with similar cycle times may be capable of considerable cycle time reduction. Therefore, a potentially better strategy is to integrate the procedures for finding a job sequence and for slowing down operations.

We use this strategy in a second heuristic, PSD, which constructs sequences by calling $NF(\cdot)$ within GENI and US, to convert a no-wait schedule into a blocking schedule. This slowing down changes the processing times of operations, which requires updating of the P and D matrices. A detailed description now follows.

HEURISTIC PARALLEL SLOWING DOWN (PSD)

Step 0. See heuristic SSD.

Step 1. Use GENI to find an initial sequence σ for problem N. During this procedure, for each subsequence δ , call $NF(\delta)$

Table 1. Relative heuristic percentage errors vs. lower bound.

m	n	Min	Mean	Max	CPU
5	25	1.19	4.04	8.73	14.30
5	50	1.25	5.75	9.44	60.79
5	100	0.82	2.33	4.16	381.03
5	250	0.77	3.03	4.94	1919.14
10	25	2.53	6.16	9.23	32.09
10	50	2.61	5.17	8.72	137.62
10	100	0.98	2.78	4.32	641.72
10	250	1.14	2.89	4.49	2431.42
15	25	1.67	4.41	7.46	73.41
15	50	1.70	4.24	8.26	273.59
15	100	1.05	2.15	3.69	730.10
15	250	0.91	2.36	4.09	3110.04
20	25	1.33	3.81	8.28	104.73
20	50	0.95	2.41	6.13	383.66
20	100	1.12	2.94	4.45	977.37
20	250	0.92	1.82	3.13	3822.30

and update P and D . Set $z^* = C_B(\sigma)$, $\sigma^* = \sigma' = \sigma$ and $j = 1$. *Step 2.* Starting from sequence σ' , use US and GENI with job j to find a sequence σ'' in the neighborhood of σ' . During this procedure, for each subsequence δ found by GENI, call $NF(\delta)$ and update P and D . If $C_B(\sigma'') < z^*$, then set $\sigma^* = \sigma' = \sigma''$ and $z^* = C_B(\sigma'')$.

Step 3. See Heuristic SSD.

5. COMPUTATIONAL RESULTS

Table 1 shows the best computational results obtained using both heuristics SSD and PSD, and a lower bound using Lagrangean relaxation and one- and two-machine decomposition approaches for the resulting subproblems (Abadi et al. 1997). Because we use the Pascal version of GENIUS, all codes are written in Pascal. All testing is performed on a SUN SPARCstation 20 model 50. The third, fourth, and fifth columns, respectively, show the minimum, mean, and maximum relative percentage errors between the best upper and lower bounds on cycle time, over five randomly generated instances. Processing times are generated from a uniform distribution over the integers $1, \dots, 100$. The mean CPU time in seconds is shown in the last column.

We believe that our solution of problems with 20 machines and 250 jobs with a mean relative error of less than 2% represents a valuable improvement in the solvability of the blocking flowshop problem. Of course, part of this relative error is approximation of the lower bound, so the actual heuristic error may be less. In view of the comment by Baker (1992) that no algorithm is known that quickly solves all instances of the classical flowshop makespan problem with five machines and 50 jobs, we believe that, as a result of our work, the blocking flowshop problem is now more tractable than the classical problem.

6. CONCLUSIONS

In this paper we describe heuristic methods for minimizing the steady-state cycle time to produce a minimal part set repetitively in an m machine blocking flowshop. Computational results indicate that our heuristic procedures are effective for problem instances at least one order of magnitude larger than those solved in earlier studies. We believe that these heuristics are the first that will prove effective for industrial size problems.

The key idea in our paper is slowing down operations to make a connection between the computationally solvable no-wait flowshop problem and the previously almost intractable blocking flowshop problem. Our work suggests the usefulness of making similar connections in scheduling research to achieve better leverage with existing computationally successful algorithms. Among possible topics for future research is the use of similar ideas for other scheduling environments, in particular jobshops and openshops with blocking. It is also useful to consider other objectives, such as minimizing the total cost of slowing down and speeding up operations to achieve a given throughput rate.

ACKNOWLEDGMENTS

This research was supported in part by Natural Sciences and Engineering Research Council of Canada (Grant no. OGP0104900) and by the Summer Fellowship Program (Fisher College of Business, The Ohio State University). The authors wish to thank Professor Gilbert Laporte

(University of Montreal) for supplying a copy of the GENIUS program, and Professor Marc Posner (The Ohio State University) for providing helpful comments on an earlier draft of this paper.

REFERENCES

- Abadi, I. N. K., N. G. Hall, C. Sriskandarajah. 1997. Minimizing cycle time in a blocking flow-shop. Working Paper, Fisher College of Business, The Ohio State University, Columbus, OH.
- Baker, K. R. 1992. *Elements of Sequencing and Scheduling*. Amos Tuck School of Business Administration, Dartmouth College, Hanover, NH.
- Garey, M. R., D. S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman & Company, San Francisco, CA.
- Gendreau, M., A. Hertz, G. Laporte. 1992. New insertion and postoptimization procedures for the traveling salesman problem. *Oper. Res.* **40** 1086–1094.
- Hall, N. G., C. Sriskandarajah. 1996. A survey of machine scheduling problems with blocking and no-wait in process. *Oper. Res.* **44** 510–525.
- Karabati, S., P. Kouvelis. 1996. Cyclic scheduling in flow lines: modeling observations, effective heuristics and a cycle time minimization procedure. *Naval Res. Logist.* **43** 211–231.
- McCormick, S. T., M. L. Pinedo, S. Shenker, B. Wolf. 1989. Sequencing in an assembly line with blocking to minimize cycle time. *Oper. Res.* **37** 925–935.
- Orlin, J. B. 1993. A faster strongly polynomial minimum cost flow algorithm. *Oper. Res.* **41** 338–350.
- Piehl, J. 1960. Ein beitrag zum reinhenfolgeproblem. *Unternehmensforschung* **4** 138–142.